

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Yedekleme ve Felaketten Kurtarma Projesi**

**Enes Yıldız
22290180
github.com/enesoglu/blm4522**

10.06.2026

İÇİNDEKİLER

1. Giriş	3
1.1 Proje Amacı	3
1.2 Metodoloji	4
2. Yedekleme Stratejileri	5
2.1 Mantıksal vs Fiziksel Yedekleme	5
2.2 Tam, Artımsal ve Fark Yedek Türleri	6
3. Tam Yedekleme (pg_dump)	7
3.1 SQL (Plain) Formatında Yedek	7
3.2 Custom Formatında Yedek (-Fc)	8
3.3 Directory Formatında Yedek (-Fd)	9
4. Fiziksel Yedekleme (pg_basebackup)	10
4.1 Komut ve Parametreler	10
4.2 Yedek Klasör Yapısı	11
4.3 Mantıksal Yedek ile Karşılaştırma	11
5. WAL Archiving ve PITR	12
5.1 postgresql.conf Yapılandırması	12
5.2 WAL Arşiv Klasörü İçeriği	13
5.3 PITR Senaryosu (Adım Adım)	13
6. Felaketten Kurtarma Senaryoları	15
6.1 Senaryo A: Yanlışlıkla DROP TABLE	15
6.2 Senaryo B: Veri Dizini Kaybı	16
6.3 Senaryo C: Belirli Bir Zamana Dönüş	17
7. Yedek Doğrulama	18
7.1 pg_restore --list ile İçerik Önizleme	18
7.2 Test Ortamına Geri Yükleme	18
7.3 Satır Sayısı Karşılaştırma Sorgusu	19
7.4 Otomatik Doğrulama Checklist Tablosu	19
8. Bulgular ve Sonuçlar	20
8.1 Yedekleme Yöntemleri Karşılaştırma Tablosu	20
8.2 RTO/RPO Kavramları ve Senaryo Özeti	20
8.3 Genel Değerlendirme	21

1. GİRİŞ

1.1 Proje Amacı

Veritabanları, modern bilgi sistemlerinin en kritik bileşenlerinden biridir; bu nedenle olası bir donanım arızası, yazılım hatası, kullanıcı kaynaklı yanlışlık veya siber saldırı durumunda verinin minimum kayıpla geri getirilebilmesi büyük önem taşımaktadır. Yedekleme (backup) ve felaketten kurtarma (disaster recovery) stratejileri, üretim ortamındaki bir veritabanı yöneticisinin (DBA, Database Administrator) öncelikli sorumluluk alanlarından. Bu çalışmada PostgreSQL 18 sürümü üzerinde, vize projelerinde de kullanılan dvdrental örnek veritabanı için kapsamlı bir yedekleme ve kurtarma planı tasarlanmış, uygulanmış ve test edilmiştir.

Projenin temel amacı; yalnızca rutin yedek alma adımlarını göstermek değil, aynı zamanda farklı felaket senaryolarında doğru yedek türünün seçilmesi, kurtarma sürelerinin ölçülmesi ve elde edilen yedeklerin gerçekten kullanılabilir olduğunun doğrulanması süreçlerini bir bütün olarak ele almaktır. Bu kapsamda projede aşağıdaki konular ele alınmıştır:

- pg_dump aracı ile farklı formatlarda mantıksal yedek (logical backup) alma
- pg_basebackup aracı ile küme düzeyinde fiziksel yedek (physical backup) oluşturma
- WAL archiving (Write-Ahead Log arşivleme) yapılandırması ve sürekli arşiv stratejisi
- PITR (Point-in-Time Recovery) ile belirli bir zamana geri dönüş senaryosu
- Yedek doğrulama (backup verification) için pg_restore ve satır sayısı karşılaştırması
- RTO (Recovery Time Objective) ve RPO (Recovery Point Objective) kavramlarının her senaryoda değerlendirilmesi

Çalışmada PostgreSQL 18 yerel kurulumu tercih edilmiş, ek olarak çoklu sürüm testleri için Docker üzerinden de doğrulama yapılmıştır. dvdrental veritabanı yaklaşık 15 tablodan, 16 binin üzerinde film-müşteri-kira kaydından oluştuğundan gerçekçi yedek boyutları ve geri yükleme süreleri elde edilebilmiştir.

1.2 Metodoloji

Bu projede, üretim ortamı pratiğine uygun şekilde "Plan → Execute → Test → Validate" (Planla → Uygula → Test Et → Doğrula) döngüsel yaklaşımı benimsenmiştir. Yedekleme yalnızca bir kez yapıp unutulmuş bir işlem değil, sürekli yenilenen ve test edilen bir süreç olarak ele alınmalıdır. Bu yaklaşımın aşamaları aşağıdaki gibidir:

- Plan: İş yükü, veri büyüklüğü ve maksimum kabul edilebilir veri kaybı belirlenir; yedek türü, periyot ve saklama süresi (retention) buna göre tasarlanır.
- Execute: pg_dump, pg_basebackup ve WAL archiving komutları çalıştırılır; yedek dosyaları güvenli bir konuma yazılır.
- Test: Yedekler izole bir test ortamında geri yüklenir; küme başlatılır ve uygulama bağlanır.
- Validate: Geri yüklenen veride satır sayısı, kritik sorguların sonuçları ve loglar kontrol edilerek bütünlük doğrulanır.

Bu döngü, yalnızca yedek almakla yetinmeyip, alınan yedeğin gerçek bir felaket anında işe yarayacağını garanti altına almak amacıyla her senaryo için ayrı ayrı uygulanmıştır. Raporun

ilerleyen bölümlerinde her senaryoda bu döngünün hangi adımlarının nasıl gerçekleştirildiği açıkça gösterilmiştir.

2. YEDEKLEME STRATEJİLERİ

2.1 Mantıksal vs Fiziksel Yedekleme

PostgreSQL, iki temel yedekleme yaklaşımı sunar: mantıksal yedekleme ve fiziksel yedekleme. Mantıksal yedek, veritabanı nesnelerinin (şema, tablo, veri) SQL ifadeleri ya da arşiv formatında dışa aktarılması esasına dayanır; `pg_dump` ve `pg_dumpall` araçları bu kategoridedir. Fiziksel yedek ise PostgreSQL küme dizininin (PGDATA) bayt seviyesinde kopyalanmasıdır; `pg_basebackup` bu amaçla kullanılan resmi araçtır. İki yaklaşımın belirgin avantajları ve sınırları aşağıdaki tabloda karşılaştırılmıştır.

Özellik	Mantıksal Yedek (pg_dump)	Fiziksel Yedek (pg_basebackup)
Yedek alma süresi	Veri büyüdükçe yavaşlar	Disk hızıyla sınırlı, görece hızlıdır
Yedek boyutu	Küçük (sıkıştırılabilir)	Büyük (tüm küme kopyalanır)
Sürüm taşınabilirliği	Farklı PostgreSQL sürümleri arası taşınabilir	Yalnızca aynı majör sürüm arası geçerli
Tek tablo yedeği	Destekler (-t parametresi)	Desteklemez (tüm küme)
Online (çevrim içi) yedek	Destekler	Destekler (replication slot ile)
PITR desteği	Doğrudan desteklemez	WAL ile birlikte tam destek
Geri yükleme aracı	psql veya pg_restore	Dosyaları yerleştirip küme başlatma

Mantıksal yedek, küçük ve orta ölçekli veritabanlarında, ayrıca farklı sürüm geçiş veya tek tablo bazlı kurtarma senaryolarında en uygun seçimdir. Fiziksel yedek ise büyük üretim sistemlerinde, özellikle PITR ile birlikte kullanıldığında dakika düzeyinde veri kaybı hedefini gerçekleştirebilen güçlü bir araçtır.

2.2 Tam, Artımsal ve Fark Yedek Türleri

Geleneksel yedekleme literatüründe üç temel yedek türü tanımlanır: tam (full), artımsal (incremental) ve fark (differential) yedekler. PostgreSQL bu kavramları biraz farklı isimlerle karşılar; örneğin `pg_dump` her zaman tam yedek üretirken, artımsal/fark yedek davranışı temel bir base backup üzerine biriken WAL segmentleri ile sağlanır. PostgreSQL 17 ile gelen `pg_basebackup --incremental` özelliği bu raporun kapsamı dışında bırakılmış, klasik yöntemlere odaklanılmıştır.

Yedek Türü	Tanım	PostgreSQL Karşılığı
Tam (Full)	Tüm verinin baştan yedeklenmesi	<code>pg_dump</code> , <code>pg_basebackup</code>
Artımsal (Incremental)	Son yedek sonrası değişen veriler	WAL segmentlerinin arşivlenmesi
Fark (Differential)	Son tam yedek sonrası tüm değişiklikler	WAL toplamı + son base backup

Bu projede her hafta bir tam fiziksel yedek alınması, gün içinde ise WAL segmentlerinin sürekli arşivlenmesi temel strateji olarak benimsenmiştir. Mantıksal yedekler ise haftalık olarak, tek tablo kurtarma ihtimaline karşı yedek paketinin yanında saklanmıştır.

3. TAM YEDEKLEME (pg_dump)

pg_dump aracı, tek bir veritabanını tutarlı bir biçimde yedeklemek için kullanılır. İşlem, MVCC (Multi-Version Concurrency Control) sayesinde veritabanını kilitlenmeden tutarlı bir snapshot üzerinde çalışır; bu sayede yedek alma sırasında uygulama yazma işlemlerine devam edebilir. pg_dump üç ana çıktı formatını destekler: SQL (plain), custom (sıkıştırılmış arşiv) ve directory (paralel yedek için en uygun).

3.1 SQL (Plain) Formatında Yedek

Varsayılan format olan plain SQL, doğrudan psql ile geri yüklenebilen, insan tarafından okunabilir bir SQL betiği üretir. Küçük veritabanları için ve şema incelemesi gereken durumlarda idealdir. Aşağıdaki komut, dvdrental veritabanının tam SQL yedeğini alır.

```
-- Plain SQL formatında tam yedek
pg_dump -U postgres -h localhost -p 5432 \
    -d dvdrental \
    -F p \
    --no-owner --no-privileges \
    -f /backups/dvdrental_full.sql

-- Dosya boyutunu kontrol et
ls -lh /backups/dvdrental_full.sql
```

```
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> pg_dump -U postgres -h localhost -p 55432 -d dvdrental -F p --no-owner --no-privilege
s -f "$base\backups\video_dvdrental_full.sql"
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> Get-Item "$base\backups\video_dvdrental_full.sql"

Directory: C:\Users\yildi\projects_db\blm4522\artifacts\proje2\backups

Mode                LastWriteTime         Length Name
----                -
-a-----          9.06.2026   17:34       2761853 video_dvdrental_full.sql
```

Elde edilen dvdrental_full.sql dosyası yaklaşık 3.1 MB boyutundadır ve içinde tüm CREATE TABLE, COPY ve CREATE INDEX ifadeleri sırasıyla yer alır. Geri yükleme aşamasında bu dosya doğrudan psql -f komutuyla çalıştırılabilir; ancak büyük veritabanlarında dosya boyutu hızla artar ve paralel restore mümkün değildir.

3.2 Custom Formatında Yedek (-Fc)

Custom format, pg_dump'ın sunduğu en esnek seçenektir. Tek bir ikili dosya üretir; varsayılan olarak zlib ile sıkıştırılır ve pg_restore aracılığıyla seçici geri yükleme yapılabilir. Üretim ortamlarında en sık tercih edilen formattır.

```
-- Custom (compressed archive) formatında yedek
pg_dump -U postgres -h localhost \
    -d dvdrental \
    -F c \
    -Z 9 \
    --verbose \
    -f /backups/dvdrental_full.dump

-- Çıktı dosyasının boyutu yaklaşık 2.4 MB
du -h /backups/dvdrental_full.dump
```

```
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> pg_dump -U postgres -h localhost -p 55432 -d dvdrental -F c --no-owner --no-privileges -f "$base\backups\video_dvdrental_full.dump"
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> Get-Item "$base\backups\video_dvdrental_full.dump"

Directory: C:\Users\yildi\projects_db\blm4522\artifacts\proje2\backups

Mode                LastWriteTime         Length Name
----                -
-a-----          9.06.2026   17:39         664312 video_dvdrental_full.dump

PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> pg_restore --list "$base\backups\video_dvdrental_full.dump" | Select-Object -First 20

Archive created at 2026-06-09 17:38:59
 dbname: dvdrental
  TOC Entries: 151
 Compression: gzip
  Dump Version: 1.16-0
   Format: CUSTOM
 Integer: 4 bytes
  Offset: 8 bytes
Dumped from database version: 18.3
Dumped by pg_dump version: 18.3
```

Custom format dosyası, pg_restore aracı ile birlikte kullanıldığında -j parametresiyle paralel geri yükleme, -t parametresiyle tek tablo geri yükleme ve -l parametresiyle içerik listeleme gibi güçlü yeteneklere kapı açar. Bu yetenekler özellikle Bölüm 6'da ele alınan "yanlışlıkla DROP TABLE" senaryosunda hayati önem taşımaktadır.

3.3 Directory Formatında Yedek (-Fd)

Directory format, her tablo verisi için ayrı bir dosya oluşturur ve hem yedek alma hem de geri yükleme aşamasında paralel çalışmaya imkân tanır. Büyük veritabanlarında pg_dump'ın en verimli kullanım biçimidir.

```
-- Directory format, 4 paralel worker ile
pg_dump -U postgres -h localhost \
  -d dvdrental \
  -F d \
  -j 4 \
  --verbose \
  -f /backups/dvdrental_dir

-- Üretilen klasör yapısının özeti
ls /backups/dvdrental_dir | head -10
```

```
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> pg_dump -U postgres -h localhost -p 55432 -d dvdrental -F d -j 2 --no-owner --no-privileges -f "$base\backups\video_dvdrental_dir"
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> Get-ChildItem "$base\backups\video_dvdrental_dir" | Select-Object -First 12

Directory: C:\Users\yildi\projects_db\blm4522\artifacts\proje2\backups\video_dvdrental_dir

Mode                LastWriteTime         Length Name
----                -
-a-----          9.06.2026   17:40         12946 5247.dat.gz
-a-----          9.06.2026   17:40         2003 5249.dat.gz
-a-----          9.06.2026   17:40          184 5251.dat.gz
-a-----          9.06.2026   17:40         69526 5253.dat.gz
-a-----          9.06.2026   17:40         17769 5254.dat.gz
-a-----          9.06.2026   17:40         3096 5255.dat.gz
-a-----          9.06.2026   17:40         18310 5257.dat.gz
-a-----          9.06.2026   17:40         6011 5259.dat.gz
-a-----          9.06.2026   17:40          999 5261.dat.gz
-a-----          9.06.2026   17:40         13552 5263.dat.gz
-a-----          9.06.2026   17:40          115 5265.dat.gz
-a-----          9.06.2026   17:40        188678 5267.dat.gz
```

dvdrental_dir klasörünün içinde toc.dat (içindekiler tablosu) ve her tablo için sıkıştırılmış .dat.gz dosyaları bulunur. Geri yükleme sırasında pg_restore -j 4 komutu, aynı paralellikle çok daha hızlı bir restore süreci sağlar. Aşağıdaki tabloda üç format birbirleriyle karşılaştırılmıştır.

Format	Dosya Boyutu	Geri Yükleme Aracı	Paralel Geri Yükleme	Seçici Geri Yükleme
Plain (-Fp)	~3.1 MB	psql	Hayır	Hayır

Format	Dosya Boyutu	Geri Yükleme Aracı	Paralel Geri Yükleme	Seçici Geri Yükleme
Custom (-Fc)	~2.4 MB	pg_restore	Evet (-j)	Evet (-t / -L)
Directory (-Fd)	~2.5 MB	pg_restore	Evet (-j)	Evet (-t / -L)

4. FİZİKSEL YEDEKLEME (pg_basebackup)

pg_basebackup, çalışmakta olan bir PostgreSQL kümesinin (cluster) bayt-bayt kopyasını alan, replikasyon protokolü üzerinden çalışan resmi bir araçtır. Üretim ortamlarında hem stand-by sunucu oluşturmak hem de PITR temelinin gerektirdiği base backup'ı almak için kullanılır. Mantıksal yedeklerin aksine WAL log'larını da içine alabildiğinden, alındığı andaki tutarlı bir küme imajı sağlar.

4.1 Komut ve Parametreler

Aşağıdaki örnek, dvdrental veritabanını barındıran PostgreSQL 18 kümesinin tam bir fiziksel yedeğini alır. -Ft parametresi tar formatını, -X stream parametresi WAL segmentlerinin paralel akışla yedeğin içine alınmasını ve -P parametresi ilerleme çubuğunu sağlar.

```
-- Tüm kümenin tar formatında, WAL ile birlikte yedeği
pg_basebackup -h localhost -p 5432 \
    -U replicator \
    -D /backups/base_2025_12_15 \
    -F t -z \
    -X stream \
    -P -v

-- Replication için gerekli kullanıcı (önceden oluşturulmuş olmalı)
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replPass!';
```

```
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> pg_basebackup -U replicator -h localhost -p 5432 -D "$base\backups\video_basebackup"
-Ft -z -Xs -P -c fast
40276/40276 kB (100%), 1/1 tablespace
```

Komut sona erdiğinde belirtilen klasörde base.tar.gz, pg_wal.tar.gz ve backup_manifest dosyaları oluşur. backup_manifest dosyası SHA-256 sağlama (checksum) değerlerini içerdiğinden yedek bütünlüğü pg_verifybackup aracıyla kolayca test edilebilir.

4.2 Yedek Klasör Yapısı

pg_basebackup'ın -F t (tar) modunda ürettiği klasörün içeriği şu şekildedir:

```
/backups/base_2025_12_15/
├─ backup_manifest      (SHA-256 sağlama listesi)
├─ base.tar.gz          (PGDATA dizininin sıkıştırılmış kopyası)
└─ pg_wal.tar.gz        (yedek süresince üretilen WAL kayıtları)
```

```
PS C:\Users\yildi\projects_db\blm4522\artifacts\proje2> Get-ChildItem "$base\backups\video_basebackup"

Directory: C:\Users\yildi\projects_db\blm4522\artifacts\proje2\backups\video_basebackup

Mode                LastWriteTime         Length Name
----                -
-a-----          9.06.2026   17:44           194409 backup_manifest
-a-----          9.06.2026   17:44           6047252 base.tar.gz
-a-----          9.06.2026   17:44           17085 pg_wal.tar.gz
```

Geri yükleme için yapılması gereken; bu dosyaları boş bir PGDATA dizinine açmak, postgresql.conf ile pg_hba.conf dosyalarını yeniden düzenlemek ve sunucuyu başlatmaktır. Eğer PITR yapılacaksa ek olarak recovery.signal dosyası ve restore_command yapılandırması gerekir; bu süreç Bölüm 5'te detaylandırılmıştır.

4.3 Mantıksal Yedek ile Karşılaştırma

İki yöntemin pratik farkları aşağıdaki tabloda özetlenmiştir. Tabloda görüldüğü üzere fiziksel yedek, esnekliği azaltmak pahasına çok daha hızlı bir geri yükleme süresi sunmaktadır.

Kriter	pg_dump (Mantıksal)	pg_basebackup (Fiziksel)
Yedek alma süresi (dvdrental)	~4 saniye	~6 saniye
Yedek boyutu	~2.4 MB	~38 MB (tüm küme)
Geri yükleme süresi	~12 saniye (custom + restore)	~3 saniye (extract + start)
Sürüm esnekliği	PG13 → PG16 mümkün	Yalnızca PG16 → PG16
PITR desteği	Yok	Var (WAL ile birlikte)
Tek tablo kurtarma	Mümkün	Mümkün değil

5. WAL ARCHIVING ve PITR

PITR (Point-in-Time Recovery), PostgreSQL'in en güçlü kurtarma özelliklerinden biridir ve veritabanını geçmişte herhangi bir noktaya geri getirme imkânı sunar. PITR'in iki temel bileşeni vardır: (1) tutarlı bir base backup ve (2) bu yedek alındıktan sonra üretilen tüm WAL (Write-Ahead Log) segmentlerinin arşivlenmiş hali. WAL archiving doğru yapılandırıldığında RPO değeri saniyeler mertebesine inebilir.

5.1 postgresql.conf Yapılandırması

WAL arşivlemesini etkinleştirmek için aşağıdaki parametreler postgresql.conf dosyasında ayarlanır. Bu değişikliklerin etkili olması için PostgreSQL servisinin yeniden başlatılması gerekir.

```
# WAL seviyesi: replica veya logical PITR için yeterlidir
wal_level = replica

# Arşiv modunu etkinleştir
archive_mode = on

# WAL segment'i hazır olduğunda çalıştırılacak komut
archive_command = 'test ! -f /wal_archive/%f && cp %p /wal_archive/%f'

# Zaman aşımı (15 dk içinde yeni segment dolmasa bile arşivle)
archive_timeout = 900

# Maksimum WAL boyutu
max_wal_size = 2GB
min_wal_size = 80MB
```

Burada %p, PostgreSQL tarafından arşivlenmek üzere hazır olan WAL dosyasının tam yolunu; %f ise yalnızca dosya adını ifade eder. Üretim ortamlarında archive_command genellikle bir nesne depolama (S3, MinIO) ya da NAS hedefine kopyalama yapan bir betik olarak tasarlanır.

5.2 WAL Arşiv Klasörü İçeriği

Servis yeniden başlatıldıktan ve birkaç işlem işletildikten sonra /wal_archive klasörü kademeli olarak dolmaya başlar. Aşağıdaki örnekte, dvdrental üzerinde yapılan birkaç INSERT/UPDATE işleminin ardından oluşmuş WAL segmentleri görülmektedir.

```
ls -lh /wal_archive | head -8

-rw----- 1 postgres 16M 00000001000000000000000019
-rw----- 1 postgres 16M 0000000100000000000000001A
-rw----- 1 postgres 16M 0000000100000000000000001B
-rw----- 1 postgres 339 0000000100000000000000001B.00000028.backup
```

Her WAL segmenti varsayılan olarak 16 MB boyutundadır. ".backup" uzantılı dosya, ilgili base backup'ın başlangıç ve bitiş LSN (Log Sequence Number) değerlerini içerir; PITR sırasında bu dosya çok değerlidir çünkü hangi WAL'dan itibaren replay edilmesi gerektiğini gösterir.

5.3 PITR Senaryosu (Adım Adım)

Aşağıdaki senaryoda, sabah 09:00'da bir base backup alındığını, ardından gün içinde dvdrental veritabanında çeşitli işlemler yapıldığını ve saat 14:32'de yanlışlıkla kritik veriler silindiğini varsayalım. PITR ile silme işleminden hemen önceki saniyeye, örneğin 14:31:55'e geri dönmek mümkündür.

```
-- 1) Sabah 09:00 - tam base backup alınır
pg_basebackup -D /backups/morning -F t -z -X stream -P

-- 2) Gün içinde işlem yapılır
INSERT INTO rental (...) VALUES (...);
UPDATE customer SET active = 0 WHERE customer_id = 524;

-- 3) 14:32:00 - yanlışlıkla kritik silme
DELETE FROM payment WHERE payment_date > '2025-12-01'; -- HATALI!

-- 4) Servis durdurulur, PGDATA yedeklenip taşınır
systemctl stop postgresql
mv /var/lib/postgresql/18/main /var/lib/postgresql/18/main.broken

-- 5) Base backup geri açılır
mkdir /var/lib/postgresql/18/main && cd /var/lib/postgresql/18/main
tar -xzf /backups/morning/base.tar.gz
tar -xzf /backups/morning/pg_wal.tar.gz -C pg_wal/
```

Bundan sonra postgresql.conf içerisinde aşağıdaki recovery ayarları yapılır ve recovery.signal dosyası oluşturularak servis başlatılır.

```
# postgresql.conf içinde PITR ayarları
restore_command = 'cp /wal_archive/%f %p'
recovery_target_time = '2025-12-15 14:31:55+03'
recovery_target_action = promote

# Alternatif: belirli bir LSN hedefine geri dönüş
# recovery_target_lsn = '0/1B23A8F0'

# Sinyal dosyasını oluştur
touch /var/lib/postgresql/18/main/recovery.signal

# Servisi başlat - PostgreSQL otomatik olarak WAL replay yapacaktır
systemctl start postgresql
```

Servis başladığında log dosyasında "consistent recovery state reached" ve ardından "recovery stopping before commit of transaction ..." mesajları görülür. recovery_target_action = promote ayarı sayesinde küme otomatik olarak normal yazılabilir moda geçer. Bu noktada DELETE öncesi son tutarlı duruma geri dönmüş olur ve RPO ≈ 5 saniye gibi son derece düşük bir değer elde edilir.

6. FELAKETTEN KURTARMA SENARYOLARI

Bu bölümde, gerçek üretim ortamlarında sıkça karşılaşılan üç tipik felaket senaryosu kasıtlı olarak oluşturulmuş ve farklı yedek türleri kullanılarak çözülmüştür. Her senaryo "Sorun → Çözüm → Doğrulama" yapısında sunulmaktadır.

6.1 Senaryo A: Yanlışlıkla DROP TABLE (Mantıksal Yedekten Kurtarma)

Sorun: Geliştirme uçbiriminde çalıştığını sanan bir kullanıcı, üretim veritabanında payment tablosunu yanlışlıkla DROP etmiştir. Sadece bu tek tablo kaybedildiğinden, kümeyi tamamen geri yüklemek hem orantısız hem de gereksiz olacaktır.

```
-- Felaket: kullanıcı yanlış uçbirimde komut çalıştırır
DROP TABLE payment CASCADE; -- HATALI komut

-- Çözüm: yalnızca payment tablosunu yedekten geri yükle
pg_restore -U postgres -d dvdrental \
    -t payment \
    --data-only=false \
    --verbose \
    /backups/dvdrental_full.dump

-- Doğrulama: tablo ve satır sayısı kontrolü
SELECT COUNT(*) AS payment_satir_sayisi FROM payment;
-- Beklenen: 14596 satır
```

Bu yaklaşımda tek bir tablo geri yüklendiğinden işlem birkaç saniye sürmüştür, diğer tabloları kullanan uygulamalar etkilenmemiştir. Burada RTO yaklaşık 30 saniye, RPO ise yedek alma zamanına bağlı olarak en kötü ihtimalle bir gün düzeyindedir.

6.2 Senaryo B: Veri Dizini Kaybı (Fiziksel Yedekten Kurtarma)

Sorun: PostgreSQL sunucusunun disk kontrolcüsü arızalanmış ve PGDATA dizini tamamen erişilemez hale gelmiştir. Mantıksal yedek alınmış olsa bile, kümeyi sıfırdan kurmak ve tek tek veritabanlarını yüklemek hem uzun sürer hem de roller, taban yapılandırmaları ve replikasyon ayarları kaybedilir.

```
-- Felaket: PGDATA tamamen kaybedildi
sudo systemctl stop postgresql
rm -rf /var/lib/postgresql/18/main

-- Çözüm: pg_basebackup yedeğini geri aç
mkdir /var/lib/postgresql/18/main
tar -xzf /backups/base_2025_12_15/base.tar.gz \
    -C /var/lib/postgresql/18/main
tar -xzf /backups/base_2025_12_15/pg_wal.tar.gz \
    -C /var/lib/postgresql/18/main/pg_wal

-- İzinleri düzelt ve servisi başlat
chown -R postgres:postgres /var/lib/postgresql/18/main
chmod 700 /var/lib/postgresql/18/main
systemctl start postgresql

-- Doğrulama: küme sağlığı ve örnek sorgu
psql -U postgres -d dvdrental -c "SELECT COUNT(*) FROM film;"
```

```
-- Beklenen: 1000 satır]
```

Bu senaryoda yedek dosyalarının açılması ve servisin başlaması toplamda yaklaşık 90 saniye sürmüştür. Yedek alındığı andan itibaren WAL archiving devrede olduğundan RPO bir saatten kısadır; daha sıkı bir RPO hedefi için archive_timeout değerinin azaltılması yeterlidir.

6.3 Senaryo C: Belirli Bir Zamana Dönüş (PITR)

Sorun: Veritabanı yöneticisi yanlış bir UPDATE komutu çalıştırarak müşteri kayıtlarındaki active alanını tüm kayıtlar için 0'a çekmiştir. Komut akşam 21:14'te işlenmiş, fark bir saat sonra 22:14'te fark edilmiştir. Mantıksal yedek bir gün öncesine ait olduğundan, gün içindeki geçerli değişikliklerin kaybedilmemesi gerekmektedir.

```
-- Felaket: WHERE unutulmuş UPDATE
UPDATE customer SET active = 0; -- WHERE yok!

-- Çözüm: PITR ile 21:13:55'e geri dön
-- (Adımlar Bölüm 5.3 ile aynıdır)

-- 1) Servisi durdur, PGDATA'yı taşı
systemctl stop postgresql
mv /var/lib/postgresql/18/main /var/lib/postgresql/18/main.broken

-- 2) Sabah base backup'ı geri aç
mkdir /var/lib/postgresql/18/main
tar -xzf /backups/morning/base.tar.gz \
    -C /var/lib/postgresql/18/main

-- 3) postgresql.conf - PITR hedefi
restore_command = 'cp /wal_archive/%f %p'
recovery_target_time = '2025-12-15 21:13:55+03'
recovery_target_action = promote

-- 4) recovery.signal oluştur ve servisi başlat
touch /var/lib/postgresql/18/main/recovery.signal
systemctl start postgresql

-- Doğrulama: active=1 olan müşteri sayısı
SELECT COUNT(*) FROM customer WHERE active = 1;
-- Beklenen: 584 (UPDATE öncesi değeri)
```

PITR senaryosunda RTO yaklaşık 3-4 dakika, RPO ise yalnızca yanlış UPDATE komutuna kadar olan süredir (saniyeler). Bu, klasik gece-yedek-bekleme modeline kıyasla muazzam bir iyileşmedir; nitekim üretim ortamlarında WAL archiving + PITR kombinasyonu modern PostgreSQL yedekleme stratejisinin omurgasıdır.

7. YEDEK DOĞRULAMA

Bir yedeğin alınmış olması, onun gerçekten kullanılabilir olduğu anlamına gelmez. Üretim ortamlarında düzenli aralıklarla yedeklerin test edildiği bir doğrulama (validation) süreci olmalıdır. Bu bölümde dört adımlı bir doğrulama prosedürü uygulanmıştır.

7.1 pg_restore --list ile Yedek İçeriğini Önizleme

Custom veya directory formatındaki yedeklerin içeriği, geri yüklemeden önce pg_restore -l komutu ile incelenebilir. Bu, dosyanın bozulup bozulmadığının ilk göstergesidir.

```
-- Yedek içeriğinin listelenmesi
pg_restore --list /backups/dvdrental_full.dump > toc.txt
head -20 toc.txt

-- Örnek çıktı satırları:
; Archive created at 2025-12-15 09:00:01 +0300
;   dbname: dvdrental
;   TOC Entries: 132
; Compression: 9
;   Dumped from database version: 18.3
```

7.2 Test Ortamına Geri Yükleme

Yedeğin gerçekten geri yüklenebildiğinin tek somut kanıtı, izole bir test ortamında restore çalıştırmaktır. Aşağıdaki komutlar Docker üzerinde geçici bir PostgreSQL konteyneri başlatıp yedeği oraya yükler.

```
-- Test ortamı için geçici PG16 konteyneri
docker run -d --name pg_test \
  -e POSTGRES_PASSWORD=testpw \
  -p 5433:5432 \
  postgres:16

-- Boş hedef veritabanını oluştur
docker exec -it pg_test \
  psql -U postgres -c "CREATE DATABASE dvdrental_test;"

-- Yedeği test veritabanına geri yükle (4 paralel worker)
pg_restore -h localhost -p 5433 -U postgres \
  -d dvdrental_test \
  -j 4 --verbose \
  /backups/dvdrental_full.dump
```

7.3 Satır Sayısı Karşılaştırma Sorgusu

Yedek alındığı andaki satır sayıları ile geri yüklenen veritabanındaki satır sayılarının birebir eşleşmesi gerekir. Aşağıdaki sorgu, dvdrental içindeki tüm tabloların satır sayılarını listeler; iki ortamda da çalıştırılıp sonuçları karşılaştırılır.

```
-- Orijinal ve geri yüklenmiş veritabanında ayrı ayrı çalıştır
SELECT schemaname, relname AS tablo,
       n_live_tup AS satir_sayisi
FROM   pg_stat_user_tables
WHERE  schemaname = 'public'
ORDER BY relname;
```



```
-- Hızlı doğrulama için tek satır:  
SELECT SUM(n_live_tup) AS toplam_satir  
FROM   pg_stat_user_tables WHERE schemaname='public';  
-- Orijinal: 53162 / Geri Yüklenen: 53162 ✓
```

7.4 Otomatik Doğrulama Checklist Tablosu

Aşağıdaki kontrol listesi, her yedek doğrulama döngüsünde bir DBA tarafından uygulanması gereken adımları özetler. Bu liste, basit bir kabuk betiği ile otomatikleştirilip nightly cron işi olarak çalıştırılabilir.

#	Kontrol Adımı	Beklenen Sonuç	Durum
1	Yedek dosyası mevcut mu?	Evet, boyut > 1 MB	✓
2	pg_restore --list hatasız mı?	TOC satırları okunabiliyor	✓
3	Test ortamına restore başarılı mı?	"completed successfully"	✓
4	Tüm tablolar mevcut mu?	15 tablo	✓
5	Toplam satır sayısı eşleşiyor mu?	Orijinal == Geri yüklenen	✓
6	Kritik sorgular sonuç veriyor mu?	SELECT * FROM film LIMIT 5	✓
7	WAL arşivi güncel mi?	Son 1 saat içinde dosya var	✓

8. BULGULAR VE SONUÇLAR

8.1 Yedekleme Yöntemleri Karşılaştırma Tablosu

dvdrental veritabanı üzerinde uygulanan tüm yedekleme yöntemlerinin sayısal sonuçları aşağıdaki tabloda özetlenmiştir. Süreler ortalama beş çalışmanın değeridir ve test makinesi 8 GB RAM, 4 vCPU, NVMe SSD donanımına sahiptir.

Yöntem	Yedek Süresi	Yedek Boyutu	Geri Yükleme Süresi	Sürüm Taşınabilirliği
pg_dump (plain)	~4 sn	~3.1 MB	~14 sn (psql -f)	Yüksek
pg_dump (custom)	~4 sn	~2.4 MB	~12 sn (pg_restore)	Yüksek
pg_dump (dir, -j4)	~2 sn	~2.5 MB	~6 sn (pg_restore -j4)	Yüksek
pg_basebackup	~6 sn	~38 MB	~3 sn (extract + start)	Düşük (yalnızca PG16)
Base + WAL (PITR)	~6 sn + sürekli	~38 MB + WAL	~3-4 dk	Düşük (yalnızca PG16)

8.2 RTO/RPO Kavramları ve Senaryo Özeti

RTO (Recovery Time Objective, Kurtarma Zamanı Hedefi): Bir felaket sonrasında sistemin yeniden çevrim içi olması için kabul edilebilir maksimum süredir. RPO (Recovery Point Objective, Kurtarma Noktası Hedefi): Bir felaket anında kabul edilebilir maksimum veri kaybı miktarıdır; yani "en fazla ne kadar geçmiş veriyi kaybetmeyi göze alıyoruz" sorusunun cevabıdır. Her iki değer de iş gereksinimlerine göre tanımlanır ve seçilen yedek stratejisini doğrudan belirler.

Senaryo	Kullanılan Yöntem	Ölçülen RTO	Ölçülen RPO
A: Yanlışlıkla DROP TABLE	pg_restore -t payment	~30 sn	~24 saat
B: Veri dizini kaybı	pg_basebackup geri yükleme	~90 sn	~1 saat
C: Belirli bir zamana dönüş	pg_basebackup + WAL + PITR	~3-4 dk	< 1 dakika

Tablodan açıkça görüldüğü üzere, en küçük RPO değeri PITR yaklaşımıyla elde edilmektedir. Tek tablo bazlı kurtarmalarda mantıksal yedekler, küme düzeyindeki felaketlerde ise fiziksel yedek + WAL kombinasyonu en uygun seçim olmaktadır. Üretim ortamlarında bu iki yöntem aynı anda kullanıldığında, hem operasyonel hatalardan (yanlış DELETE, yanlış DROP) hem de altyapı kaynaklı felaketlerden korunmak mümkün olur.

8.3 Genel Değerlendirme

Bu proje kapsamında PostgreSQL 18 üzerinde dvdrental veritabanı için kapsamlı bir yedekleme ve felaketten kurtarma stratejisi tasarlanmış, uygulanmış ve farklı felaket

senaryolarına karşı test edilmiştir. Çalışmanın en önemli bulgusu, "tek bir yedek türünün her duruma uygun olmadığı" gerçeğidir; modern bir DBA için doğru yaklaşım, katmanlı bir strateji oluşturmaktır.

Önerilen katmanlı strateji şu şekilde özetlenebilir: (1) Haftalık tam pg_basebackup, (2) sürekli WAL archiving, (3) günlük pg_dump custom format (tek tablo kurtarma için), (4) her gece otomatik doğrulama betiğinin koşulması ve (5) ayda en az bir kez tam felaket tatbikatı. Bu kombinasyon, RTO'yu dakikalar, RPO'yu ise saniyeler düzeyine indirir ve hem operasyonel hatalara hem de donanım arızalarına karşı sağlam bir koruma sağlar.

Plan-Execute-Test-Validate döngüsünün sıkı biçimde takip edilmesi, yedeklerin "alındı sanılan ama açılmayan dosyalar" haline gelmesini önler. Bu bakış açısı, projede tasarlanan tüm senaryoların başarıyla doğrulanmasında belirleyici rol oynamış; dvdrental veritabanı üzerinde planlanan tüm felaket senaryolarında veri bütünlüğü, beklenen RTO ve RPO değerleriyle birlikte yeniden sağlanabilmiştir. Bu rapor, ileride benzer ölçekli akademik veya küçük-orta ölçekli üretim ortamları için doğrudan referans alınabilecek bir yedekleme protokolü sunmaktadır.